



RUNTIME FEEDDOWNLOADER PRO

Advanced User Manual

AUGUST 2026

ECOSYSTEM RUNTIME | DIGITAL CORE

Chapter 1: Introduction and philosophy

1.1 What is Runtime FeedDownloader Pro?

It helps to start from the problem the software solves.

Thousands of podcast episodes are published and listened to every day. Over time a fair share of that material vanishes: the presenter stops paying for hosting, the distribution platform closes, the server holding the audio files is switched off. An episode you heard three years ago may now be gone for good — not because anyone deleted it deliberately, but because nobody kept a copy.

Runtime FeedDownloader Pro exists for that reason. It is not just a podcast downloader; it is an application for the systematic preservation of audio published through RSS feeds. It is meant for people who treat sound as a document, and therefore file it with the same care they would give a text or a photograph: archivists, newsrooms, radio stations, researchers, methodical listeners.

1.2 Who it is for

Four typical ways of using it.

- **The archivist** wants to pull down the whole catalogue of a long-running podcast before it disappears. What they need is a system that remembers what it already has, never duplicates anything and certifies the integrity of every file.
 - **The radio producer** keeps the library on a shared NAS. They need a tool that works on network paths without seizing up, files things predictably and produces CSV inventories for the rest of the team.
 - **The publisher** keeps a local copy of the network's podcasts, exports metadata to the editorial system and watches the state of the archive over time.
 - **The listener** wants their podcasts on disk, in order, without depending on a connection or ending up with truncated files.
-

1.3 The database-first philosophy

What separates FeedDownloader Pro from a generic downloader is where the truth lives.

Almost every download tool reasons the same way: it looks at the files on disk, compares them with the RSS feed and fetches whatever is missing. The flaw is obvious — a disk is not a reliable source. Files get moved, renamed, corrupted, deleted by accident. Shift your podcast folder from `C:\Podcast` to `D:\Archive` and the tool loses every reference, then cheerfully re-downloads the entire catalogue.

FeedDownloader Pro keeps the truth somewhere else. At the centre of every operation sits an SQLite database that records each episode it has met or downloaded: the source URL, the file name on disk, the down-

load date, the SHA-256 checksum of the content, the audio metadata. The database is the software's memory, and it stays accurate even when files move.

Four practical consequences follow from that choice.

1. **No duplicates.** Analyse the same feed ten times over and the episodes already on record will not go back into the queue.
 2. **Resilience to moves.** The archive can migrate to another disk or to a NAS without losing its history.
 3. **Persistent state.** If the program is closed halfway through a batch of 300 episodes, the history is intact on reopening and the missing episodes are still listed as pending.
 4. **A record of operations.** Every downloaded file carries its date, its source URL and its cryptographic checksum.
-

1.4 Three underlying technical choices

Beyond the database-first design, the program rests on three decisions you notice directly in use.

Network resilience

Downloading hundreds of audio files from the internet is an operation that fails often, and in various ways. Servers get overloaded, connections drop, transfers are cut off halfway. The software tackles this on three fronts.

- **Retries with growing delays.** When a download fails for a transient reason, the program does not immediately try again: it waits one

second, then two. There are three attempts in all; if the server explicitly asks for a delay (the `Retry-After` header), the program honours it up to a maximum of sixty seconds.

- **Stall detection.** A stuck download is worse than a failed one: it hangs there and never frees the queue. The program watches the data flow and, if not a single byte arrives for sixty seconds, it closes the transfer, counts it as a failed attempt and moves on.
- **.part files with resume.** Every transfer writes to a temporary file with a `.part` extension. Only once the file is complete and its size matches what the server declared does it take its final name. If a transfer is interrupted, the `.part` stays on disk and the next attempt **resumes from where it stopped** instead of starting over (an HTTP Range request with an If-Range validator). Orphaned `.part` files can be cleared from **Settings → Advanced → Clean temporary files**.

URL safety

The program handles URLs that arrive from outside, namely from feeds. A carefully crafted address can point at resources on the local network — the router, the NAS, an internal server — and turn the application into a way of reaching them. This is the family of attacks known as SSRF, *server-side request forgery*.

Every URL therefore passes through validation on several levels: syntax checking, acceptance of HTTP and HTTPS only, rejection of known internal hostnames, rejection of private and reserved address ranges (RFC 1918, loopback, link-local) and a fresh check of the resolved IP address on every connection and every redirect. The whole thing is automatic and invisible in use.

Network paths

The archive can live on a network drive. Handling SMB paths — the ones used by NAS devices, Windows servers and shared folders — is a classic source of freezes in desktop applications: if the remote disk stops responding, the interface locks up until the operating system gives up. FeedDownloader Pro checks the path outside the graphics process and gives up on its own after eight seconds. The interface stays responsive either way.

1.5 How to read this manual

The manual covers the whole of the program, from installation to the corners you rarely visit. You do not need to read it in order: the chapters stand on their own.

To get a feel for the software, start with chapter 4, which follows a complete run from analysing a feed to the archived file. If you already know your way around, go straight to whichever chapter you need.

Chapter 2: Installation and first launch

2.1 System requirements

Runtime FeedDownloader Pro is a desktop application built on Electron. It is self-contained: it does not need Node.js, .NET or Java installed on the machine, because everything it uses travels inside the package.

Minimum requirements

Operating system	Minimum version	Architecture
Windows	10 (build 1903) or Windows 11	64-bit (x64)
Linux	Ubuntu 22.04 LTS, Debian 11, Fedora 36 or equivalent	64-bit (x64)

Recommended hardware

- **Memory:** 4 GB, better 8 GB for large archives with several downloads running in parallel
- **Disk:** 200 MB for the program, plus the space the audio archive needs
- **Network:** broadband, at least 10 Mbps for parallel downloads to be worth having

A note for Linux. The software ships as an `.AppImage` (self-contained, runs on any recent distribution without installation) and as a `.deb` for Debian- and Ubuntu-derived distributions.

A note on macOS. There are no official macOS builds and none are planned. The source code can be compiled on a Mac, but the build is the responsibility of whoever runs it and is not supported.

2.2 Installing on Windows

1. Download `Runtime-FeedDownloader-Pro-1.5.0.exe` from the releases page.
2. Double-click the downloaded file.
3. If the **Windows protected your PC** warning appears (SmartScreen), press **More info** and then **Run anyway**. The warning is normal for programs distributed outside the Microsoft Store without a commercial signature: it concerns the file's reputation, not its contents.
4. Choose the installation folder and press **Install**.
5. When it finishes you will find a desktop shortcut and a Start menu entry.

Where the files go. The installation is per user and needs no administrator privileges: the program goes to `C:\Users\[YourName]\AppData\Local\Programs\Runtime FeedDownloader Pro\` unless you choose otherwise during setup. The database lives elsewhere, in `C:\Users\[YourName]\AppData\Roaming\Runtime FeedDownloader Pro\`, so uninstalling the program does not touch the archive.

2.3 Installing on Linux

1. Download `Runtime-FeedDownloader-Pro-1.5.0.AppImage` , or the `.deb` package for Debian and Ubuntu.
2. Make the AppImage executable, either way:
 - from the desktop: right-click the file → Properties → Permissions → tick the option to run it as a program;
 - from a terminal: `chmod +x Runtime-FeedDownloader-Pro-1.5.0.AppImage`
3. Launch it with a double-click or from a terminal: `./Runtime-FeedDownloader-Pro-1.5.0.AppImage`

For the Debian package: `sudo dpkg -i Runtime-FeedDownloader-Pro-1.5.0.deb` , or the distribution's graphical package manager.

Desktop integration. To get an icon in the applications menu you can use **AppImageLauncher**, available in most distribution repositories, which registers AppImages with the system.

A note for sandboxed environments. Under **Flatpak** or other environments that restrict filesystem access, the program may not see SMB network paths. In that case, check that the share is mounted and reachable from the file manager before launching the application.

2.4 The first launch

At startup a brief splash screen appears with the program name and two buttons, **Start Application** and **Skip**: the second takes you straight to the

interface, the first does the same once the animation ends. The screen returns every time you open the program.

After that the software is ready. No configuration is needed, no account, no licence key. The interface opens with the URL field at the top and an empty episode list.

Files created on first launch. A single file appears in the user data folder:

- `feeddownloader.sqlite` , the database. It holds the download history, the episode metadata, the list of feeds and the preferences (destination folder, parallel downloads, speed and size caps, refresh interval).

Do not delete this file.

The interface language is the exception: the program remembers it alongside the display preferences rather than inside the database, and you choose it from **Settings** → **General**.

2.5 Updates

Updates always require your explicit consent. The path has three steps.

1. At startup, and on demand from **Settings** → **Advanced** → **Check for Updates**, the program checks whether a newer version exists. If it finds one, the **Update available** banner appears in the top bar and stays there until you act on it (you can close it with the ×, and it returns on the next launch or the next check).
2. The download starts **only** when you press **Download** on the banner, or **Download update** in the settings. Nothing is fetched on the program's own initiative.

3. Once the download completes, the banner becomes **Update ready**: press **Restart & install** and the program closes, applies the update and starts again. Installation does not happen on its own either, and it is not triggered when you quit the application.

The update replaces the program files and leaves the archive alone. The first time the new version starts, the **What's new in this version** window sums up what changed.

A suggestion. Before a major version jump (say from 1.4.x to 1.5.x), it is worth copying `feeddownloader.sqlite` somewhere safe.

Chapter 3 describes the interface element by element.

Chapter 3: A tour of the interface

3.1 Anatomy of the main window

The window falls into four zones.

- **Command bar, at the top.** It holds the URL field, the analysis button, the button for changing the destination folder, access to the command palette and the settings icon. When an update exists, its banner appears here too (chapter 2, section 2.5).
- **Feed sidebar, on the left.** Home to the library of saved feeds, the Archive tab, the sync controls and, at the bottom, the destination path. You adjust its width by dragging the right edge.
- **Main area, in the centre.** Shows the episodes of the selected feed, with the podcast header, the filter bar and the list.
- **Download panel, on the right.** It opens by itself when a batch of downloads starts. When it is closed, a round button in the bottom right corner brings it back.

On first launch, with the library empty, the main area shows a guide panel explaining how to add your first feed and where to set the destination folder. It disappears on its own once the first feed enters the library.

3.2 The command bar

URL field. This is where you paste the podcast's RSS address. It accepts URLs pointing at XML or RSS files and supports dragging: you can take a link from the browser and drop it on the field.

Analyze. Contacts the address, reads the feed and fills the episode list. When it finishes, the feed takes its place in the sidebar for good. It usually takes one to five seconds, depending on the size of the feed and the network.

Folder icon. Opens the picker for changing the download destination folder, without going through the settings.

Settings icon. Opens the settings panel, even while a download is running (chapter 10).

3.3 The feed sidebar

The sidebar is the heart of the library: the feeds it holds survive closing the program.

The Feeds and Archive tabs

Two tabs sit at the top. **Feeds** shows the library and is the default view. **Archive** opens the table of every episode downloaded across the whole library (section 3.10).

How to read a library row

Each feed shows its cover art, the title declared in the RSS feed and the date of the last sync. When there are episodes that have never been downloaded, the **TO DOWNLOAD** badge appears with a count (for example **3 TO DOWNLOAD**). The count is based on the episodes' unique identifier, the GUID, as recorded in the database: an episode counts as new only if its GUID has never been seen before. The badge disappears once all the new episodes have been downloaded.

A click on the row loads that feed's episodes into the main area.

Adding a feed

Paste the URL into the field at the top and press **Analyze**. The feed is saved to the library and stays available in later sessions.

Search and sorting

The search field filters feeds by name as you type, which helps once the library grows. The sort button arranges the feeds alphabetically; press it again to restore the original order.

Synchronisation

Hover over a feed and the sync icon appears: it re-reads the feed from the server and updates the list with any new episodes.

The **Sync All** button, at the top of the sidebar, refreshes the whole library in parallel. While it works, each cover shows its own state through an icon: spinning while in progress, a green tick on success, a red marker if the feed does not respond. The button reports progress (**Syncing... 3/7**). The results stay visible for two and a half seconds, then fade.

At the bottom: the destination path

The last row of the sidebar shows the destination folder shortened to its last two components (for example `Documents / Podcasts`). A click opens it in the system file manager. To change it, use the folder icon in the command bar or **Settings** → **Download**.

Resizing

Width is adjusted by dragging the right edge: 240 pixels minimum, 640 maximum, 360 by default. The size you choose is remembered.

Connection state

The sidebar header reports whether the program is **Connected** or **Offline**. When the network is down, a notice at the top reminds you that downloads stay put until it comes back; once the connection returns, the program picks up again and checks whether new episodes appeared in the meantime.

3.4 The episode list

Once a feed is selected, the main area fills with its episodes.

Feed header

At the top you get the cover art, the podcast title and the episode count. This is also where the commands that act on the whole feed live (section 3.7).

What each row shows

- **Title** of the episode as it appears in the feed.
- **Date** of publication.
- **Duration**, when the feed declares it.
- **Status**, with the matching tag (section 3.5).

File size is not shown in the list: you read it in the detail panel, where before the download it is the value declared by the feed and afterwards the real size of the file on disk.

The filter bar

A bar under the header lets you narrow the list.

- **Text search.** Filters by keyword, looking in the episode title and description. AND logic applies: every term you type must be present.
- **Status.** Three quick buttons: **All**, **Not Downloaded**, **Downloaded**.
- **Date.** Two fields, **From** and **To**, to limit the publication range.
- **Duration.** Minimum and maximum in minutes.
- **Sort.** Five criteria: feed order (the default), newest date, oldest date, longest duration, shortest duration.

Change feed and all the filters reset.

Multiple selection

You can pick several episodes and download them as a block.

- **Ctrl+click** adds or removes a single episode from the selection.
- **Shift+click** selects the range between the last episode chosen and the one you clicked.

- The tick box appears on hover for unselected episodes and stays permanently visible on selected ones.

With at least one episode selected, **Download Selected (N)** appears in the header. The selection clears when you change feed and once a download starts.

3.5 Episode states

Every episode carries a status indicator. Reading them is the quickest way to see how the archive stands.

State	Where you see it	Meaning
To download	NEW tag in the list	The episode is in the feed and has never been downloaded.
Queued	<i>queued</i> in the download panel	Waiting its turn.
Paused	<i>paused</i> in the download panel	Suspended by you; resumes from where it stopped.
In progress	Animated progress bar	Transfer under way, with percentage and speed.
Error	Error summary in the download panel	All automatic attempts failed.

State	Where you see it	Meaning
Downloaded	ARCHIVED tag in the list	The file is on record in the database, from this session or an earlier one.

The **ARCHIVED** tag follows directly from the database-first design. Re-analyse a feed you have already worked through and almost every episode will come back as archived: the program knows it has them. Only those published after your last download show up as **NEW**.

3.6 Commands on a single episode

Hover over a row and the commands for that episode appear on the right. They vary with its state.

Always available

- **Copy title:** copies the episode title to the clipboard.
- **Tick box:** used for multiple selection (section 3.4).

Episodes pending or in error

- **Download:** puts the episode in the queue.

Episodes already archived

- **Forget download:** returns the episode to pending without touching the file on disk.
- **Open Folder:** opens the file manager at the file's location.

To fetch an archived episode again, go through the detail panel, where **Re-download** puts the file back in the queue and overwrites the existing one.

A plain click on the row opens the detail panel (section 3.9). Ctrl+click and Shift+click are reserved for multiple selection and open nothing.

3.7 Commands on the whole feed

These commands sit in the feed header, above the filter bar, and act on many episodes at once.

Download All queues every episode never downloaded among those **currently visible**: if a filter is active, it acts only on those. Before it starts, the program asks for confirmation ("Do you really want to download all N episodes?") and checks free space on the destination disk, warning you if there is not enough. The download panel opens by itself.

Download Selected (N) appears when there is an active selection and downloads only that.

Sync New re-reads the feed from the server and immediately queues any episodes that have just appeared, without going through the list. The free-space check applies here too.

Export M3U produces an `.m3u` playlist with the local paths of that podcast's downloaded episodes and opens the save dialog. The button is always present: if nothing from that feed is on disk yet, the program says so rather than creating an empty file.

Change Folder, the folder icon in the header, opens the destination folder picker.

To stop or suspend active downloads you use the download panel controls instead (section 3.8).

3.8 The download panel

This is where you watch and steer everything that is downloading.

Opening and closing it

It opens by itself when a batch starts. Close it and the round button in the bottom right corner brings it back. Closing the panel stops nothing.

How it is laid out

- **At the top**, the counter of completed files against the total (**47 / 312** , say), a summary of how many downloads are active and how many queued, and the × for closing the panel.
- **In the middle**, the queue: one row per download, with the episode title, the podcast name, the percentage, the current speed and a progress bar. If the server does not declare the file size, the bar becomes indeterminate and the bytes received scroll in place of the percentage. Each row carries **Pause** (which becomes **Resume** once the row is suspended) and **Cancel download**.
- **At the bottom**, the commands that apply to the whole queue: **Pause**, which suspends every transfer and turns into **Resume** while the label beside it reads **Queue paused**, and **Stop download**, which halts everything and empties the queue.

Pausing and stopping are not the same thing

Pausing, whether a single episode or the entire queue, keeps the partial `.part` file: on resuming, the transfer picks up where it left off. **Stop download** is final: it deletes the partial files, empties the queue and returns the interrupted episodes to **NEW**. Files already completed stay in the archive, naturally.

When something goes wrong

At the end of the batch, if any downloads failed, an expandable summary appears at the bottom with the missed episodes and each one's error code. Beside it is **Retry failed**, which puts them all back in the queue at once, saving you from hunting through the list.

3.9 The detail panel

A plain click on a row opens the detail panel, which slides in from the right below the command bar. It closes on its own when you move to another feed. Ctrl+click and Shift+click do not open it: they belong to multiple selection.

It contains:

- **Basic metadata:** publication date, declared duration, size stated in the feed.
- **Actions**, which change with the episode's state: Download, Re-download, Forget download, Open Folder.

- **Archive data**, only for episodes already downloaded: download date, real size, bitrate, sample rate, file name on disk and the SHA-256 checksum recorded at download time.
 - **Source link**: the URL of the audio file in the feed, with a button to copy it.
 - **Episode notes**: the descriptive text published in the feed, stripped of its HTML.
-

3.10 The Archive view

You reach it from the **Archive** tab in the sidebar. Where the episode list shows one feed at a time, here every downloaded episode appears in a single table, whichever podcast it came from.

- **Search**: looks in the episode title, the podcast name and the date, both as you see it on screen and in ISO form (**2026-08-18**).
- **Filter by podcast**: a dropdown narrows the table to one show.
- **Sorting**: by download date, publication date, size or bitrate.
- **Statistics**: at the top, the number of files, the number of distinct podcasts and the space used.
- **Show in folder**: hover over a row to open the file manager at that file.

The table refreshes itself as each download completes.

3.11 The command palette (Ctrl+K)

The palette reaches the main functions without a mouse. It opens with **Ctrl+K** from anywhere in the program, including during a download, and presents a search field in the centre.

Type to filter, move with the **↑↓ arrows**, confirm with **Enter**, leave with **Esc** without running anything.

Results come in groups.

- **Quick actions:** five commands that are always there — open settings, sync all feeds, add feed (which puts the cursor in the URL field), open archive, go back to feeds.
 - **Jump to feed:** with the field empty it shows the first feeds in the library; as you type it filters them by title. Choose one and it loads into the main area.
 - **Episodes (current feed):** as you type, the palette also searches the episode titles of the open feed. Choose one and the list is filtered on that title.
-

3.12 The notification area icon

Close the window with the **×** and the program does not quit: it shrinks into the notification area, near the clock. This is deliberate, because downloads and the automatic check for new episodes carry on with the window closed.

A click on the icon shows or hides the window. The right-click menu has two entries, in English because the system provides them: **Show** brings

the window back to the front, **Quit** closes the program and stops any active downloads.

In practice: for a large archive, start the batch, close the window and let the machine work.

Chapter 4 walks through a complete run, from the first analysis to the archived file.

Chapter 4: Your first archive, step by step

4.1 What we are going to do

This chapter follows a whole run: from a podcast's address to a folder of tidy audio files on disk. The scenario is the most common one — downloading a programme's complete catalogue for the first time.

It is worth reading once from beginning to end. Once you have the hang of it, opening a new archive takes under a minute.

4.2 Finding the feed address

Everything starts from the podcast's RSS address. An RSS feed is a text file in XML format that the publisher puts online to list the available episodes: every podcast has one, even when it is not advertised.

Where to look:

- **On the podcast website.** An orange icon with waves, or the words RSS, Feed, Subscribe, Podcast Feed. Clicking it usually opens the XML file in the browser: the address in the bar is the one you need.
- **In a listening app.** Pocket Casts, Apple Podcasts and the like often show the RSS link in the programme information, sometimes tucked under Share.

- **On the hosting platform.** Spreaker, Podbean, Buzzsprout and similar services publish the feed address on the podcast page or in the producer's dashboard.
- **With a search engine.** Searching `[podcast name] RSS feed` often gets you there first time.

A valid RSS address usually ends in `.xml` or `.rss`, or contains `feed`, `rss` or `podcast` in the path. For example:
`https://www.example.com/feed.xml` ,
`https://feeds.spreaker.com/podcast/12345` ,
`https://anchor.fm/s/abc123/podcast/rss` .

4.3 Preparing the destination folder

Before analysing the feed it is worth deciding where the files will go, and giving yourself a structure from the start.

```
D:\Podcast Archive\  
├─ My Podcast\  
├─ Technology Podcast\  
└─ Radio Talk Show\  

```

The program creates the subfolder named after the podcast inside your chosen destination, and saves the files there under the names the template decides (chapter 8).

To set the destination:

1. Press the **folder** icon in the command bar at the top, or open **Settings** → **Download** and use the icon next to **Download Path**.

2. Choose the folder and confirm.
3. The path stays visible at the bottom of the sidebar; a click opens it in the file manager.

If the archive lives on a NAS or a network drive, read chapter 7 before going further: those paths need a little more care.

4.4 Analysing the feed

With the address ready and the folder set:

1. Paste the address into the **URL field** at the top.
2. Press **Analyze**, or simply **Enter**.
3. The list fills up. A podcast of 200 or 300 episodes takes a few seconds; catalogues past the thousand mark can need fifteen or twenty, because the XML file to read is big.

If the analysis fails, in order of likelihood: the address contains a stray space or character; the feed is temporarily unreachable, which is worth testing in a browser; the server answers with an HTTP code that the program reports in the error message, typically **403 Forbidden** for feeds that demand particular headers.

4.5 Reading the results

Once the analysis is done, the list shows the catalogue.

Three things are worth a look. The **episode count**, at the top of the list: a programme running for a few years easily has a few hundred. The episodes already **ARCHIVED**, if the feed had been worked through before: the database knows those and will not fetch them again. And the episodes **without duration or size**, which simply do not declare them in the feed: they download the same as any other, with no consequences.

4.6 Starting the download

There are two routes.

The whole catalogue. Press **Download All**: the program asks for confirmation, checks free space and queues every episode never downloaded. Note one useful detail: it acts on the episodes **visible at that moment**, so with a filter active it downloads only those. Downloads run in parallel, three at a time unless **Parallel Downloads** says otherwise (chapter 10).

A few episodes only. Select them with **Ctrl** and a click on each, or with **Shift** for a range, and press **Download Selected (N)** in the feed header.

4.7 Following progress

The **download panel** opens by itself on the right and shows each episode with its percentage, speed and estimated time. As an order of magnitude: two hundred episodes at 64 kbps take between two and three gigabytes.

The rows in the list update in real time. There is no need to keep the window open: close it and the program stays in the notification area and carries on working.

During the batch the software handles retries after network errors, stalls and the verification of each completed file by itself. If the computer suspends, transfers are cut off: on waking, stall detection closes the frozen transfers and the retry cycle usually recovers, but any episode that had already used up its three attempts ends in error. Put those back in the queue with **Retry failed**.

4.8 Checking the archive

When the panel declares the batch complete, three things remain.

Check the errors. The summary at the bottom of the download panel lists the missed episodes with their codes. The most frequent is `EPISODE_NOT_FOUND`, meaning a file pulled from the server before we could get it. To requeue every failure in one go there is **Retry failed**.

Export the inventory. **Settings** → **Archive** → **Export Inventory (CSV)** gives you the full list of archived episodes with SHA-256 checksums, sizes and metadata (chapter 9).

Take a look at the disk. The files sit in the destination folder, named according to the template (chapter 8). If you find `.part` files, those transfers were interrupted or suspended: requeue the same episodes and they resume from where they stopped. Orphaned ones can be cleared from **Settings** → **Advanced** → **Clean temporary files**.

4.9 Keeping the archive up to date

This is where the database-first design earns its keep: the program always knows what it holds.

For a feed already in the library, select it and press the sync icon, or use **Sync All** for the whole library. New episodes appear with the **NEW** tag and the feed shows the **TO DOWNLOAD** badge in the sidebar; the ones you already have stay **ARCHIVED**. At that point **Download All** takes only the new ones. Alternatively, **Sync New** does both at once: it re-reads the feed and immediately queues what is missing.

For a feed not yet in the library, paste the address into the URL field and press **Analyze**: it joins the library and the list shows straight away what is missing.

The same episode is never downloaded twice. The automatic check for new episodes runs every six hours out of the box (chapter 10 and section 5.9).

Chapter 5 goes into feed management and OPML files in detail.

Chapter 5: Managing feeds

5.1 What an RSS feed is

An RSS feed is an XML document a podcast publishes so that programs can read its list of episodes. When a new instalment comes out, the publisher adds an entry to that document; readers re-read it now and then and notice the addition.

For FeedDownloader Pro the feed is the source of everything: it carries the episode list, the addresses of the audio files, the metadata (title, date, duration, description, cover art) and the general information about the programme.

You do not need to know its internal structure to use the software. It does help, though, in understanding why data is sometimes missing from the list: nearly always it is the feed that fails to declare it.

5.2 Well-made feeds and awkward ones

Standards compliance, in the podcast world, is an aspiration rather than a rule.

A well-formed feed. It respects RSS 2.0 or Atom, declares title, link, date and audio address with its MIME type, and may add iTunes or Podcast

Index tags for duration, cover art and seasons. The program reads it without fuss.

An incomplete feed. Optional fields such as duration, size or episode artwork are missing. The files download the same, but some information stays blank in the list.

A feed with unreachable audio. The document reads fine, but the file addresses point at resources that are gone. This happens often with abandoned programmes or ones that moved elsewhere. The episodes end in error after the download attempt.

Password-protected feeds. Some private podcasts ask for HTTP Basic credentials. You can put them straight into the address:
`https://user:password@www.example.com/feed.xml` .

5.3 What happens when you press Analyze

In sequence:

1. **Address check.** Correct syntax, and clearance from the checks against SSRF attacks (chapter 10).
2. **Request to the server.** The program identifies itself with an ordinary user agent and gives up after fifteen seconds of silence.
3. **Reading the document.** It reads RSS or Atom and tolerates the common deviations: undeclared encoding, missing tags, invented namespaces.
4. **Comparison with the archive.** Each episode is looked up in the database through its unique identifier, the GUID; when the feed does not declare one, the audio file address stands in for it.

5. **Filling the list.** Every episode appears with its own state.
 6. **Entry into the library.** The feed joins the sidebar if it was not there already; if it was, the episode count is updated.
-

5.4 The feed library

Analysed feeds stay saved in the sidebar: there is no need to paste the addresses again at every launch.

What each row shows

Cover art, title, date of the last sync and, when there are episodes never downloaded, the **TO DOWNLOAD** badge with a count. The count comes from comparing the feed's GUIDs with the ones already on record. A click loads the episodes into the main area.

Removing a feed

Hover over the row and a bin icon appears, which asks for confirmation. Removal deletes the feed from the library but **does not touch the audio files already downloaded**, nor their records: the episodes remain visible in the Archive view.

Search and sorting

The field at the top filters by name as you type. The sort button puts the feeds in alphabetical order and, pressed again, restores the original order.

On privacy. The library lives only in the local database. The program does not send your feeds to anyone.

5.5 Importing feeds from an OPML file

OPML is the format podcast applications use to exchange subscription lists. If you have a library in Pocket Casts, Overcast, AntennaPod or similar, you can export it and bring it in here.

1. Open **Settings** → **Archive**, section **Data & Portability**.
2. Press **Import Feeds (OPML)** and choose the `.opml` file.
3. The program reads it and adds the feeds it finds to the library.

If the import fails, it is worth opening the file in a text editor and looking for lines like `<outline type="rss" xmlUrl="...">`: some applications use proprietary variants of the format and leave exactly those out.

5.6 Exporting the library to OPML

Exporting is for keeping a copy of the list, moving it to another installation, or following the same programmes in a listening app.

1. Open **Settings** → **Archive**, section **Data & Portability**.
2. Press **Export Feeds (OPML)** and choose a name and location.

The resulting file can be read by any application that supports the standard.

5.7 Very large feeds

Long-running podcasts and radio archives easily reach thousands of episodes.

RFC 5005 pagination. Many platforms publish only recent instalments in the feed and break the rest into pages linked to each other with `rel="next"`. The program follows those links by itself and reassembles the catalogue into a single list. Each page goes through the same security checks as an address typed by hand. The number of pages followed stops at twenty: a safety ceiling against feeds that refer to one another endlessly, which on enormous catalogues means the oldest part may be left out.

Analysis takes longer. Two thousand episodes, or a feed split across many pages, can need fifteen to thirty seconds. That is normal.

The list stays light. Even with thousands of entries, only the rows visible on screen are drawn.

Space. Two thousand episodes at fifty megabytes apiece make a hundred gigabytes: worth counting before you start.

5.8 Working with many feeds

There is no limit to the number of podcasts in the library, and they all stay available from one session to the next.

A click on a feed in the sidebar loads its episodes immediately. At startup no feed is selected: the program does not remember the last one open, it shows the library and waits.

To refresh them: the sync icon on an individual row re-reads that feed; **Sync All** re-reads them all in parallel, showing the state on each cover and flagging the news at the end with the **TO DOWNLOAD** badge.

For scheduled refreshing, with nothing for you to do, see the next section.

5.9 Automatic refresh

The program checks the feeds by itself: at startup, then at regular intervals, and every time the connection returns after an outage.

You choose the interval in **Settings** → **General** → **Automatic Feed Refresh**.

Option	Behaviour
Disabled	No cyclical checks; the one at startup remains.
6 hours (default)	Every feed re-read every six hours.
12 hours	Re-read every twelve hours.
24 hours	Re-read once a day.

The change takes effect at once, with no restart. The count starts again from the program launch.

Automatic refresh **downloads nothing**: it only looks for new episodes. When it finds some, it sends a system notification with the summary, in the interface language. The notification is clickable: it brings the program to the front and, if the news concerns a single podcast, opens that feed directly. To download them you then use the normal commands.

Chapter 6 explains how the download engine works.

Chapter 6: The download engine

6.1 How it is built

The engine works asynchronously and in parallel: instead of fetching one file at a time, it handles several transfers together through a central queue.

Four parts are involved. The **queue** collects, in order, every episode waiting. The **download workers** each take one episode from the queue and see it through, knowing nothing about the others; how many work at once is up to you. The **database manager** records the state of every transfer moment by moment. The **integrity check** calculates the SHA-256 checksum of each file as it arrives and writes it to the archive.

6.2 How many downloads in parallel

This is the setting worth tuning first. Too few and archiving crawls; too many and you saturate the connection, annoy the server or multiply the errors.

The default is **3**, a fair compromise on a domestic line.

Situation	Parallel downloads
Slow connection, or a server that throttles	1

Situation	Parallel downloads
Ordinary home line	3 (default)
Fast fibre	5
NAS behind a slow network	1

You change it in **Settings** → **Download** → **Parallel Downloads**, choosing between **1**, **3** and **5**. The change applies immediately, including to the queue in progress.

If the server objects. Several hosting services limit simultaneous connections per IP address. When **429 Too Many Requests** or **503 Service Unavailable** errors keep appearing, drop to one or two. The retry mechanism handles the failures anyway, but taking the load off fixes the problem at source.

6.3 Errors and retries

Across hundreds of files, some network errors are a certainty. The program does not press on straight away: it waits, and the wait grows with each failure.

Attempt	What happens
After the first failure	waits 1 second and retries
After the second	waits 2 seconds and retries

Attempt	What happens
---------	--------------

After the third

the episode is declared in **error**

If the server answers **429 Too Many Requests** and states a wait in the **Retry-After** header, the program honours it up to a maximum of sixty seconds. Since the partial **.part** file survives from one attempt to the next, the retry resumes from where it stopped rather than starting over.

Some errors do not deserve a second attempt, and the program recognises them:

- **File does not exist** (**404 Not Found**): retrying will not conjure it up.
- **Content is not audio**: the server answered with a web page instead of the file, which is typical of expired links that bounce to a courtesy page. The download is refused with the message *The server sent a web page, not audio*.
- **File too large**: it exceeds the **Maximum File Size** you set (chapter 10).
- **Disk full or access denied** on the destination folder.
- **Address rejected by the security checks** against SSRF attacks.

At the end of a batch, **Retry failed** requeues every episode in error with a single click.

6.4 Stall detection

There is a case worse than an error: the transfer that neither finishes nor fails. The connection stays technically open, the operating system

reports nothing, but the bytes have stopped arriving. With no defence, that download would hang there occupying a slot in the queue.

It happens mostly with servers under load that choke the bandwidth after the first bytes, with routing problems along the way and with large files served by constrained distribution networks.

Every active transfer is therefore watched. If not a single byte arrives for **sixty seconds**, the program closes the connection, **keeps the .part file** and counts what happened as a failed attempt: the next attempt resumes from where it stopped. If the stall was temporary the matter sorts itself out; if it repeats until the three attempts are used up, the episode ends in error.

6.5 .part files

During a transfer the content goes into a temporary file with a **.part** extension. The final name arrives only once the transfer is complete and the size matches what the server declared. That way a truncated audio file never appears in the destination folder wearing the proper extension.

The SHA-256 checksum is calculated straight afterwards, on the now-final file, and recorded in the archive along with size, bitrate and sample rate.

Resuming. In the event of a pause, a transient error or an interruption, a small **.part.meta** file stays alongside the **.part** holding the validator the server supplied (an ETag or Last-Modified). On the next attempt the program asks only for the missing bytes, with an HTTP **Range** request

accompanied by **If-Range** : if the remote file has changed in the meantime, the server says so and the download starts from zero. That is the guarantee against the worst risk of resuming, namely gluing together pieces of two different files.

Cleaning up. Orphaned **.part** files left over from past sessions can be cleared from **Settings** → **Advanced** → **Maintenance** → **Clean temporary files**; the function only works with downloads stopped.

Partial files sit in the same folder as the completed ones. There is no point opening them in an audio player: they are stumps, and the player will complain.

6.6 Pause, resume, stop

Pause. From the download panel you can suspend a single transfer with **Pause** on its row, or the whole queue with **Pause** at the bottom of the panel: the label **Queue paused** reports the state. Pausing keeps the partial files and the place in the queue; **Resume** carries the transfer on from the exact point where it stopped.

Stopping. **Stop download** is a different matter: it closes the active transfers in an orderly way, empties the queue and deletes the partial files. Whatever was already complete stays in the archive; the interrupted episodes go back to showing as **NEW**.

Closing the window. With the × the program retreats into the notification area and carries on downloading. The **Quit** entry in the tray menu, by contrast, closes everything and stops the transfers; the **.part** files

stay on disk, so requeuing those episodes in a later session resumes from where we were.

6.7 Speed

Each row in the panel shows the speed of its own transfer. There is no aggregate total on screen: to get a sense of the bandwidth in use, add up the active rows by eye.

Four factors determine it.

- **Your line**, which remains the ceiling.
 - **The source server**, which is often the real bottleneck: many hosting services throttle bandwidth to contain costs, and a single transfer rarely exceeds 2–5 MB/s.
 - **The number of parallel downloads**, which offsets a slow server by using several connections together.
 - **File size**: files of 20–80 MB, meaning half-hour or hour-long episodes, are the most efficient, because the time spent opening the connection counts for little against the total.
-

Chapter 7 covers paths on NAS devices and network drives.

Chapter 7: NAS, network drives and SMB paths

7.1 Why network drives are a case apart

Almost every desktop application copes well with `C:\` and `D:\` and turns unpredictable when the destination is a NAS, a Windows shared folder or an SMB volume. The reason is technical: a network drive is less dependable than a local one. The NAS may be off or asleep, the network may spike in latency, the credentials may have expired. And an operation on a path that does not answer will block the program for tens of seconds, freezing the interface.

FeedDownloader Pro is built not to be taken hostage. If your archive lives on a NAS, this chapter concerns you.

7.2 How the path is verified

When you choose a destination folder, the program checks that it exists and that it is writable. The check runs outside the process that draws the interface and has an **eight-second limit**: once that expires, the path is declared unavailable and a warning appears. The window never blocks waiting.

It is one check, not a battery of tests: the program asks the operating system whether it can write in there and takes the answer at face value. No test file is created in your folder.

The program recognises as network paths those beginning with `\\` or `//` (Windows UNC and SMB notation) and, on Linux and macOS, those under `/mnt/`, `/media/` and `/Volumes/`. On these it shows a dedicated warning when something is wrong. A drive mapped to a letter, `Z:` for instance, is treated by the system as any other disk: the writability check still happens, but the program does not recognise it as a network path and will not flag it as such.

Why eight seconds. Consumer NAS devices (Synology, QNAP, WD MyCloud) take three to six seconds to wake from standby. Eight is enough to wait for them and still short enough not to feel like a freeze.

7.3 Setting a path on a NAS

A direct UNC path. In the form `\\ServerName\ShareName\Folder` :

```
\\MYNAS\Podcast\Archive
\\192.168.1.100\media\podcast
\\NAS-SYNOLOGY\video\audio_archive
```

You can type it by hand or reach it through the picker, which on Windows browses network resources too.

A mapped drive. If the NAS is already mounted as a drive (say `Z:` pointing at `\\MYNAS\Podcast`), just choose `Z:\Archive`. It works without trou-

ble; the program simply treats it as a local disk, because at that point the operating system is handling the network.

Linux. Mounted SMB shares appear as ordinary folders (`/mnt/nas/podcast` , for example) and are used directly.

7.4 Credentials and authentication

NAS credentials are configured in the operating system, not inside FeedDownloader Pro.

Windows. Open the NAS path (`\\MYNAS\`) in File Explorer, enter the credentials when prompted and tick the option to remember them. They go into Windows Credential Manager (*Control Panel* → *Credential Manager* → *Windows Credentials*), and from then on the program reaches the share like any other application.

Linux. Mount the share with the credentials in `fstab` , or from a graphical file manager, or by hand with `mount -t cifs` .

7.5 When the network path does not answer

Faced with the unreachable-path warning, check in this order.

Is the NAS awake? Many home devices go into standby after a period of inactivity. A quick visit to the admin panel in a browser is the fastest way to find out.

Is it reachable on the network? From Command Prompt or a terminal:

```
ping 192.168.1.100
```

If it answers, basic networking works and the problem is higher up.

Does the share open? Try `\\192.168.1.100\ShareName` directly from the file manager. If it will not open there, the problem is in the NAS SMB configuration and not in the program.

Can you write to it? Create a file by hand in the destination folder. If the system refuses, the user you connect as has no write permission on that share: fix it from the NAS panel.

Is a firewall involved? SMB uses port 445, and in some cases 139. It is worth checking they are not blocked on the local network.

7.6 Getting good performance

Writing to a NAS means every byte crosses the network twice: speed depends on the LAN and on the device's write capacity, not just on your internet connection.

- **Cable beats Wi-Fi.** Wi-Fi adds latency and instability exactly where writes are concerned. For large archives, Gigabit Ethernet changes things.
- **Fewer downloads in parallel.** Writing many files together saturates the NAS I/O: one or two transfers often outperform five.

- **Mind the backups.** If the NAS runs automatic backups at fixed times, avoid overlapping them with download batches: they contend for the same disk and both slow down.
 - **Local first, network after.** For very large archives it pays to download to a fast local disk and move the files to the NAS once the batch is done.
-

Chapter 8 covers file names, templates and metadata.

Chapter 8: File organisation, templates and metadata

8.1 The problem with names

The name an audio file is published under is rarely meant to be read by a human: `ep_2024_03_15_FINAL_v2_mixdown.mp3` , `podcast-episode-187-compressed.m4a` , or even `abc123def456.mp3` . They make sense inside the producer's systems; inside an archive they are noise.

The program deals with this through **naming templates**: you define the format of the names once and every downloaded file follows it, drawing on the information in the feed.

8.2 How a template works

A template is a line of text mixing fixed parts with **tokens**, that is variables in curly brackets. Once a download finishes, each token is replaced with that episode's value.

The template `{date} - {podcast} - {title}` gives you:

```
2024-03-15 - Mario's Podcast - Episode 187: Artificial intelligence explained properly.mp3
```

The extension is not part of the template: it is added according to the format of the source file.

8.3 The available tokens

Token	What it holds	Example
<code>{title}</code>	Episode title from the feed	<code>Episode 187: AI explained properly</code>
<code>{pod-cast}</code>	Podcast name	<code>Mario's Podcast</code>
<code>{date}</code>	Publication date, <code>YYYY-MM-DD</code>	<code>2024-03-15</code>
<code>{year}</code>	Year	<code>2024</code>
<code>{month}</code>	Two-digit month	<code>03</code>
<code>{day}</code>	Two-digit day	<code>15</code>

When an episode declares no date, the time tokens become the word `unknown`. A made-up token, `{episode}` for instance, stays in the name exactly as written: the program does not recognise it and leaves it alone.

8.4 Recommended templates

The default is the title on its own:

```
{title}
```

That suits catalogues whose titles are already descriptive.

For general use it is worth putting the date first:

```
{date} - {title}
```

which produces `2024-03-15 - Episode 187: AI explained properly.mp3`. The advantage is practical: alphabetical file order matches chronological order.

If several podcasts end up in the same folder, include the programme name:

```
{podcast} - {date} - {title}
```

8.5 How names are cleaned up

Some characters are not allowed in file names: on Windows `/`, `\`, `:`, `*`, `?`, `"`, `<`, `>`, `|`. The program **removes** them from the name the template produces, without substituting anything: `Episode 12: The AI` becomes `Episode 12 The AI`.

Two less obvious cases are handled too. The Windows reserved names (`CON` , `PRN` , `AUX` and company) cannot be used as file names: if the title reduces to one of those, the program falls back on `episode` . And the full path, folder included, is kept within 250 characters: if need be, the title is shortened, leaving room for the technical suffixes the program adds while it works (`.part.meta` and the `_2` it may need when two names collide).

On long titles. Some podcasts use titles of a hundred and fifty characters. With `{title}` alone you get unwieldy names at risk of truncation; put `{date}` at the front and the name stays sortable and recognisable even when the tail is cut off.

8.6 Where to set it

The template is configured in **Settings** → **Metadata**, in the **File Naming Template** field. Below the field a preview shows the result on a sample episode, so you can see the effect before closing the settings.

8.7 One thing the template does not do

The template generates a **file name, not a folder structure**. Writing `{year}/{month}/{title}` does not create subfolders by year and month: the slashes are removed along with the other forbidden characters, and the result is a single file called `202403Title.mp3` .

The program applies exactly one piece of automatic organisation, and it concerns podcasts: inside the destination folder each programme gets its own subfolder, named after the podcast. To subdivide further by year, use different destination folders and switch between them from the folder icon in the command bar.

8.8 JSON sidecar files

Settings → **Metadata** has the **Sidecar .json Files** switch, off by default.

When it is on, a `.json` file with the same name appears beside each audio file, carrying the episode data:

```
{
  "title": "Episode 187: AI explained properly",
  "podcast": "Mario's Podcast",
  "guid": "https://example.com/ep187",
  "pubDate": "2024-03-15T08:00:00.000Z",
  "downloadedAt": "2026-08-18T09:14:22.517Z",
  "sourceUrl": "https://media.example.com/ep187.mp3",
  "filename": "2024-03-15 - Episode 187 AI explained properly.mp3"
}
```

It serves two purposes: letting scripts and external systems read the metadata without going through the database, and keeping a copy independent of the database itself, which helps if the archive ever has to be rebuilt.

8.9 ID3 tags

Also in **Settings** → **Metadata**, the ID3 tagging switch changes its label with its state: **ID3 Tagging Enabled** or **ID3 Tagging Disabled**. It is off out of the box.

When it is on, the program writes the data inside the file once the download finishes:

- **Title**: the episode title.
- **Artist** and **Album**: the podcast name.
- **Year**: the year of publication.
- **Cover art**: the episode image if the feed declares one, otherwise the podcast's.

Every widely used audio player reads these tags, so the information stays visible even if the file name changes.

MP3 only. **.m4a** , **.ogg** and **.opus** files are not modified even with the option on.

Chapter 9 covers file integrity and archive management.

Chapter 9: Integrity, statistics and archiving

9.1 Why files need verifying

A download that reaches the end is not necessarily a healthy file. A packet lost on the way, a write error, an interruption in the last second: the file is there, it weighs nearly what it should, and it turns out to be a stump the day somebody tries to listen to it. For an archive that is the worst way to fail, because the damage surfaces years later.

FeedDownloader Pro works on two levels: it checks the **size** while downloading and calculates the **SHA-256 checksum** as soon as the file is complete.

9.2 The SHA-256 checksum

SHA-256 produces, for any file, a sequence of 64 hexadecimal characters. Two identical files give the same checksum; change one bit and the checksum becomes unrecognisable. It is the standard way of saying "this file is exactly the one from back then".

For every download the program calculates the checksum and records it in the archive along with the file name and the date.

It serves three purposes. The **archive health check** (section 9.4) recalculates the checksum of the files present and compares it with the one on record, catching corruption and substitutions that happened after the download. **Checksum repair** uses the same checksum to recognise files somebody renamed by hand. And in a professional context, the checksum is verifiable proof of what that file contained at the moment it was archived.

9.3 Technical metadata

Once a download finishes the program reads the technical data **inside the file**, not from the feed, and records it.

Field	What it tells you	Example
Bitrate	Audio quality in kilobits per second	128 kbps , 320 kbps
Sample rate	Samples per second	44100 Hz , 48000 Hz
Size on disk	Real weight of the file	67.4 MB

They end up in the episode detail panel and in the CSV inventory (section 9.6).

9.4 The archive health check

Over time an archive changes behind the program's back: files moved, renamed, deleted by mistake, or ruined by a dying disk. The check compares the reality on disk with what the archive says.

You start it from **Settings** → **Archive** → **Archive Health Check**, pressing **Run Check**.

Two verifications are run on every recorded file: that it still exists where it is meant to, and — for those present — that the recalculated SHA-256 checksum matches the one from back then. Files that no longer match are flagged as corrupted, with a prompt to download them again.

Four numbers appear at the end:

Indicator	Meaning
In DB	Episodes recorded in the archive
On Disk	Files found where they were meant to be
Missing	Files not found
Disk Usage	Total space taken by the files present

When something is missing, the program lists the first five cases with podcast and file name, and offers two routes.

Repair archive (checksum search) looks through the podcast folders for files somebody renamed by hand: it calculates the checksum of the candidates and, when it finds the right one, re-links the file to the episode under its new name. It downloads nothing and touches no files: it up-

dates the archive. At the end it reports how many it re-linked and how many it could not find.

Mark as not downloaded is for the genuinely lost: it removes them from the register, so the episodes go back to **NEW** in the list and can be downloaded again normally.

That is the sensible order: try to repair first, then declare lost whatever is left.

9.5 Statistics

Settings → **Archive** carries three summary figures: **Files Downloaded**, the episodes on record; **Podcasts**, how many distinct programmes make up the archive; **Archive range**, the date of the first and the last download. They refresh every time you open the settings.

9.6 The CSV inventory

The export produces a file with one row per episode: useful for spreadsheets, editorial systems, scripts, or simply for having a list of what you hold outside the program.

You generate it from **Settings** → **Archive** → **Export Inventory (CSV)**, choosing where to save it.

Column	Contents
Podcast	Podcast name
Episode Title	Episode title
Publish Date	Publication date
Downloaded At	Download date and time
File Size (bytes)	Size in bytes
Bitrate (kbps)	Audio bitrate
Sample Rate (Hz)	Sample rate
SHA-256 Checksum	File checksum
Validation Status	See below
GUID	Episode identifier in the feed

The **Validation Status** column says what guarantees the episode was recorded with, not the outcome of the last check: it reads **OK** when a SHA-256 checksum is on record, **LEGACY** for older rows that have the file name but no checksum, and **UNKNOWN** when even that is missing.

Format. Comma-separated CSV, UTF-8 with BOM to keep Excel happy, fields in quotes when they contain commas.

9.7 Moving the archive

To transfer the files to another disk, use the built-in migration, which moves them and keeps the archive aligned as it goes.

1. Open **Settings** → **Archive** → **Migrate Archive**.
2. Choose the new folder.
3. The program moves the podcast folders and updates the default destination.
4. At the end it reports how many folders it moved and how many errors it met.

Careful: this is a move, not a copy — the files leave their original location. Best to check there is room at the destination first.

Moving to another computer. Two things are needed: the folder of audio files and the `feeddownloader.sqlite` database (chapter 2). On the new machine, install the program, put the database in the data folder and, if the audio files ended up somewhere different, use migration to realign everything.

Chapter 10 goes through the settings.

Chapter 10: Settings

10.1 How they are organised

The panel opens at any time from the cog icon at the top right, even while a download is running. The entries are divided into five tabs, in this order: **General**, **Download**, **Metadata**, **Archive**, **Advanced**. Every change is saved immediately: there is no confirm button because none is needed.

This chapter follows the tabs one by one. Templates, sidecars and ID3 tags (the Metadata tab) are covered in chapter 8; the health check, the CSV inventory and migration (the Archive tab) in chapter 9.

10.2 General

Language

The interface speaks **English** and **Italian**. The change is immediate, with no restart. The program uses a single theme, dark: there is no light theme and no control over list density.

Automatic feed refresh

Sets how often the program re-reads the feeds by itself. A check happens in any case at startup and when the connection returns after an outage.

Option	Behaviour
Disabled	No cyclical check; the one at startup remains.
6 hours (default)	A full re-read every six hours.
12 hours	Re-read every twelve hours.
24 hours	Re-read once a day.

The change applies at once. If the check finds new episodes it sends a clickable system notification, which brings the program to the front and opens the feed concerned when there is only one. No download starts by itself: the notification is a signal and nothing more (section 5.9).

Guide and release notes

The same tab holds two shortcuts: the **user guide**, which opens the built-in documentation and from there the full PDF manual, and **What's new in this version**, which shows the release notes for the installed version. The same window appears on its own at the first launch after an update.

10.3 Download

This is where the engine controls live. The internal parameters (connection timeout, number of retries, stall watching) are fixed and cannot be touched.

Download Path

The folder everything ends up in, in the **Storage** section. You choose it with the icon beside the field, which opens the system picker, and it does the same job as the folder icon in the command bar.

Parallel Downloads

How many transfers at once: **1**, **3** or **5**. The default is 3. For guidance on choosing, see chapter 6.

Download Speed Limit

Caps the total bandwidth the downloads use, so they do not take the whole line. The value is in KB/s; **0** means no limit and is the default. To give a sense of scale: **500** keeps consumption around 4 Mbps.

Maximum File Size

Rejects files larger than the stated value, expressed in MB; **0** means no limit. It guards against anomalies: sizes declared in error, or servers that send something quite other than audio. A file over the threshold becomes a definitive error, with no further attempts. When the server declares the size, the refusal comes before a single byte is written; when it does not, the transfer is stopped as soon as it passes the cap.

10.4 Metadata

This tab holds the **File Naming Template** with its preview, the **Sidcar .json Files** switch and the ID3 tagging switch, which changes its label with its state. They are described in chapter 8.

10.5 Archive

This gathers four groups: the archive statistics, **Data & Portability** with OPML import and export and the CSV inventory, the **Archive Health Check** with checksum repair, and **Migrate Archive** for moving the files to another disk. All in chapter 9, except OPML which is in chapter 5.

10.6 Advanced

Updates

The update system works **by consent**: nothing downloads and nothing installs on its own.

At startup the installed version queries the releases repository. If it finds a new version, the **Update available** banner appears in the top bar along with a system notification, but the package **is not downloaded**. The **Check for Updates** button forces the check whenever you want.

The download begins when you press **Download** on the banner or **Download update** here in the settings. Once the package is ready,

Restart & Install appears, and that does not fire by itself either: installation happens when you decide, never when you quit the program.

The states you will see, in order: *Checking for updates...*, then *You have the latest version* or *Update available: vX.Y.Z*, then *Downloading update... N%* and finally *Update downloaded – restart to install*.

Maintenance

Clean temporary files removes orphaned `.part` files left in the destination folder by interrupted sessions. It does not work while downloads are active, because those files are in use: in that case the program says so rather than proceeding. At the end it reports how many files it removed.

Danger Zone

Reset Download History clears the download history from the database, that is the memory of what has already been fetched. The program asks for confirmation. The audio files on disk **stay where they are**: after the reset the episodes go back to showing as pending, and a fresh download would overwrite them.

There is one case where it makes sense: when you want to start again with a clean history, for example after moving everything to another system or at the end of a run of tests.

10.7 How addresses are filtered (for information)

This section explains something the program does by itself and that cannot be configured.

A feed is content arriving from outside, and the addresses it contains could point at resources on your local network rather than at a public audio file: the router panel, a NAS, an internal service. This is the family of attacks called SSRF. An archiver that blindly follows the addresses in a feed becomes the instrument by which somebody knocks on your own doors.

Every address therefore passes through five filters:

1. **Syntax:** it must be a valid URL.
2. **Protocol:** only `http://` and `https://`. Things like `file://`, `ftp://`, `data:` or `javascript:` are turned away immediately.
3. **Internal names:** `localhost` and the loopback names are refused.
4. **Private and reserved addresses:** the RFC 1918 networks (`10.0.0.0/8`, `172.16.0.0/12`, `192.168.0.0/16`), loopback `127.0.0.0/8`, link-local `169.254.0.0/16`, and for IPv6 `::1` and `fc00::/7`.
5. **Verification at connection time:** the IP address the name resolved to is re-checked on every connection and every redirect. This is the filter that closes the cleverer loopholes, namely a public domain resolving to a private address, or a redirect that halfway along points inside the network.

A declared limitation. If your network has an internal podcast server reachable at a private address, those filters will block it, and there is no way to make an exception: there is no allow-list to configure. This is de-

liberate, because a configurable exception is exactly what a hostile feed would try to exploit. Content like that needs a different route — downloading it with another tool and importing it as local files, for instance.

Chapter 11 collects the most common problems and how to get out of them.

Chapter 11: Troubleshooting

11.1 How to use this chapter

Here are the problems that actually happen, described as they appear on screen rather than by their names inside the program. For each one: why it happens and what can be done.

If your case is not listed, the project's public releases page is the right place to report it, giving the steps that reproduce it and the exact message that appears.

Feeds and analysis

Connection error or timeout while analysing a feed

What you see: you press **Analyze**, a few seconds pass and a network error appears. The list stays empty.

The causes, in order of likelihood.

- **The feed server is not answering.** Try opening the same address in a browser: if it will not open there either, the problem belongs to the podcast and the only move is to try later.
- **Your connection is unstable.** Check that other sites respond.

- **A firewall or corporate proxy is blocking you.** If the feed opens at home and not at the office, you have your answer.

The program gives up after fifteen seconds of silence: a deliberate time-out, so you are not left staring at a frozen window.

The feed is read but the list is empty

- **The feed contains no episodes.** Open the address in a browser and look for `<item>` or `<entry>` tags. If they are not there, there is nothing to download.
- **The format is off-standard.** The program reads RSS 2.0 and Atom 1.0 and forgives a good deal of sloppiness, but some proprietary platforms produce documents that cannot be recognised.
- **You already have everything.** If the feed was worked through in the past, the episodes are there but carry the **ARCHIVED** tag. The **Downloaded** filter confirms it at a glance.

I only see the latest episodes, not the historical catalogue

Why: the limit is imposed by the publisher, not by the program. Many platforms expose only the last 50 or 100 instalments in the feed so as not to burden their own servers.

What the program already does: if the historical archive is published in linked pages following the RFC 5005 standard, the program follows them by itself and reassembles the catalogue, up to a maximum of twenty pages. The problem only remains when there is no pagination, or when the catalogue exceeds that ceiling.

What you can try: looking for an alternative "full feed", which some platforms offer; recovering old episodes from the podcast website; checking

whether the platform accepts address parameters along the lines of `?limit=0`.

Downloads

Many episodes end in error with "File not found on server"

Why: the episodes are still listed in the feed, but the audio files are gone. This happens often with abandoned programmes or ones that moved elsewhere.

There is no way to download a file that does not exist. If the podcast is alive, though, and the errors are numerous, it is worth telling whoever publishes it: sometimes it is a botched migration, and it gets fixed.

Worth knowing: failed episodes **are not excluded** from later batches. They stay marked as pending and a fresh **Download All** will try them again. That is deliberate, because a file can come back; if they bother you, the date filter or manual selection keeps them out.

Downloads are painfully slow

- **The server throttles bandwidth.** This is the most common case. With servers that penalise multiple connections, dropping to a single parallel download sometimes improves matters.
- **You are on Wi-Fi.** For long batches, cable makes the difference.
- **The destination disk is slow,** typically a NAS over Wi-Fi or a USB 2.0 stick. Download locally first and move afterwards.
- **That is your line.** A speed test settles it.

An episode sticks at 99% and never finishes

Why: the server stopped sending data without closing the connection. Within sixty seconds the stall watcher notices, closes the transfer and counts it as a failed attempt; the next attempt resumes from where it stopped, thanks to the `.part` file.

If it repeats until the three attempts are used up, the episode ends in error. Usually that means the file on the server is truncated or corrupted: you recognise it from the integrity check message, namely that the size received does not match the size declared.

I downloaded an `.mp3` that the player calls corrupted

Why: it should not happen, given the `.part` files and the size check. If it does, either the file on the server was already damaged, or there was a write error on the disk.

What to do, in order:

1. Open the episode detail panel with a click on the row and press **Re-download**.
2. If the file arrives broken a second time, the problem is at source: check it by opening the file address directly in a browser.
3. Run the archive health check (chapter 9): the SHA-256 verification lists every file whose content no longer matches what was recorded at download time.

From version 1.5.0 the program also refuses responses that are not audio: if the server sends a web page instead of the file, the download fails with *The server sent a web page, not audio* rather than saving something unusable. In the same way, a file over the ceiling set in **Maximum File Size** is turned away with *File exceeds the configured size cap*.

I need to suspend, not throw away

There is no need to stop everything for good: **Pause** in the download panel suspends the queue while keeping the partial files, and **Resume** restarts the transfers from the exact point they reached. The same applies to a single episode. **Stop download**, by contrast, deletes the partials: it is the right choice only when you really do want to throw the work away.

At the end of a batch, **Retry failed** requeues every episode that went wrong.

NAS and networking

"Network path unreachable" but the NAS is working

1. **Check the path character by character.** On some systems even capitals count (`\\MYNAS\podcast` is not `\\MYNAS\Podcast`).
2. **Check the credentials.** Open `\\MYNAS\ShareName` in File Explorer: if it asks for the password, it is not stored in Windows Credential Manager. Enter it and tick the option to remember it.
3. **Look at the firewall.** In *Windows Defender Firewall* → *Allowed apps* the program must be authorised.
4. **Check the SMB version.** Older NAS devices speak only SMBv1, which Windows 11 disables by default. Better to update the NAS firmware than to re-enable an insecure protocol.

Downloads to the NAS stop after a few minutes

Why: the NAS falls asleep mid-job. Certain domestic models watch only web traffic and ignore SMB connections, so they believe themselves idle while they are receiving files.

The fix is to disable power saving temporarily from the NAS panel, or to drop to a single parallel download: a continuous stream keeps the device awake better than bursts separated by pauses.

General problems

The interface responds sluggishly

- **A very large archive.** With tens of thousands of episodes on record, some operations slow down.
- **Too many parallel downloads on little memory.** Five transfers on a machine with less than 4 GB make themselves felt: better one or three.
- **The antivirus inspects every `.part`.** Many products scan every write to disk. Excluding the destination folder from real-time scanning fixes it.

The program will not start, or closes immediately

1. **Damaged database: automatic recovery.** If `feeddownloader.sqlite` turns out to be unreadable at startup, the program sets it aside, renaming it `feeddownloader.sqlite.corrupt-[date]`, and starts again with a new database. The application opens regardless.

2. **Guided restore.** At the first launch after a recovery, that is with an empty database and a `.corrupt-*` backup present, the *A backup of a damaged database was found.* dialog appears with **Attempt restore** and **Ignore**. The restore recovers feeds, archive and history from the backup as far as they are readable: the audio files are not touched and the backup is not modified. A summary at the end says how much was saved.
3. **Reinstall.** Uninstalling and reinstalling the latest version deletes neither the database nor the settings.

I have lost the database: can I recover anything?

- **If you have a backup**, just put `feeddownloader.sqlite` back in the data folder with the program closed (the path is in chapter 2).
- **If the database became corrupted**, look for `feeddownloader.sqlite.corrupt-[date]` files in the same folder: with the current database empty, the program offers the guided restore at startup.
- **If you have none of that**, the audio files are safe on disk regardless: what you lost is the memory, not the archive. Re-analysing the feeds gets you going again, bearing in mind that checksum repair will not help in this case, because the checksums lived in the very database that is gone.
- **To avoid a repeat**, copy `feeddownloader.sqlite` elsewhere now and then and export the feed list to OPML (chapter 5). Before a migration or a major update it is time well spent.

This is the end of the Runtime FeedDownloader Pro manual. For anything these pages do not cover, the reference is the project's public releases page.